

Energy-efficient Analog Integrated Circuits

PLL – Exercise 1

Giorgio Cristiano

Prof. Taekwang Jang

1. Introduction

In this exercise session we are going to explore the design procedure for a charge pump PLL (CPPLL), depicted in Figure 1. We will start from the design of the in-band contributors, namely phase detector and charge pump. Then, we will move to the VCO, and finally we will put everything together on MATLAB to better analyze the closed loop behaviour. The PLL will multiply a reference frequency of 25MHz by 80, obtaining an output frequency of 2GHz. Here we will focus on phase detector and charge pump.

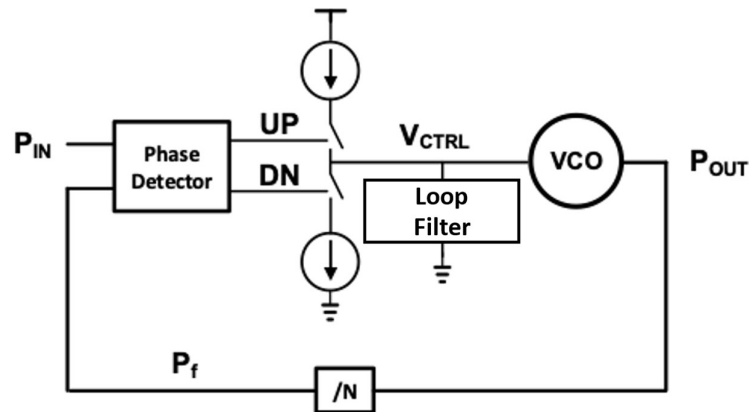


Figure 1: Block diagram of CPPLL

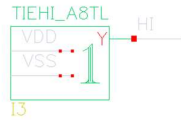
2. Design of PFD

The phase detector is a key component, capable of detecting the phase difference between two signals. In a PLL, this is necessary to translate the phase difference between reference and fed

back VCO. If the detector is also capable of detecting frequency differences (meaning that the feedback must have the same frequency as the reference, and cannot be a multiple), then it is called Phase-Frequency Detector (PFD).

In this design we will use a PFD based on two FFs. In a new library called PHDETECTOR, create a new cellview, call it PFD, and draw the schematic depicted in Fig.2.

Primary function is to provide a stable, constant logic "1" (High) to the input of another digital gate. For gate oxide protection and mitigate antenna effects



DLY: delay cells that minimizes the dead zone issue

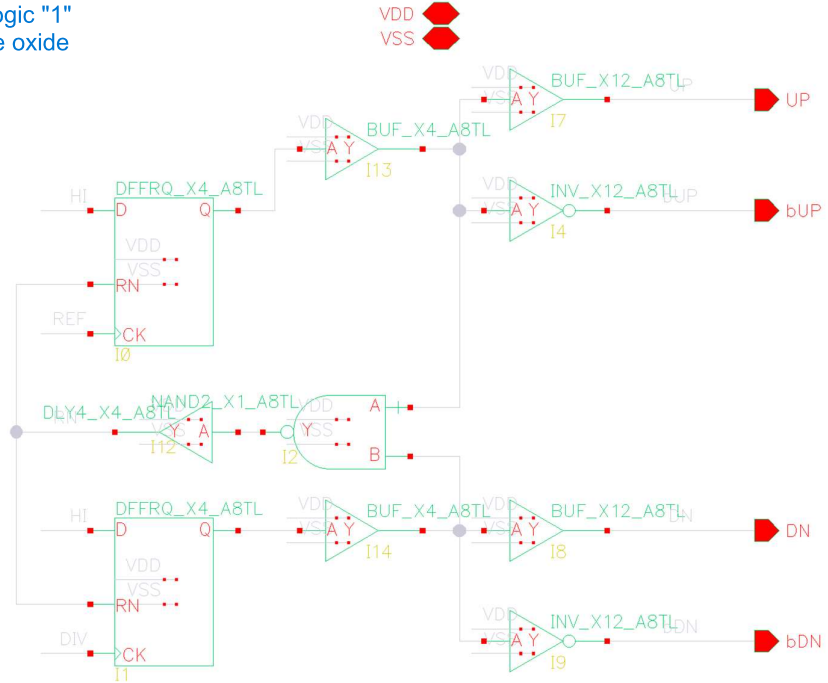


Fig 2: Schematic of PFD

Let's now test this PFD. Create a symbol for the PFD cellview. Then, create a new cellview called zTEST_PFD, and in the schematic instantiate the PFD just created. Add a 20fF loading capacitor at each at the outputs; this is used to mimic the loading of the block, as in normal operation it will be connected to the charge pump. Fix VSS at 0V, and VDD at 1.2V. For REF and DIV, we want to create a testbench in which the phase delay for the two signals can be varied. Thus, the REF signal is created through a vpulse, as shown in Fig. 3.1. While the **DIV** signal will have a parametric delay, as shown in Fig. 3.2.

Divided Clock

CDF Parameter	Value	Display	CDF Parameter	Value	Display
Frequency name for 1/period		off	Frequency name for 1/period		off
Noise file name		off	Noise file name		off
Number of noise/freq pairs	0	off	Number of noise/freq pairs	0	off
DC voltage		off	DC voltage		off
AC magnitude		off	AC magnitude		off
AC phase		off	AC phase		off
XF magnitude		off	XF magnitude		off
PAC magnitude		off	PAC magnitude		off
PAC phase		off	PAC phase		off
Voltage 1	0 V	off	Voltage 1	0 V	off
Voltage 2	1.2 V	off	Voltage 2	1.2 V	off
Period	1/25M s	off	Period	1/25M s	off
Delay time	25n s	off	Delay time	25nt (dly_deg / (360*25M))	off
Rise time	5p s	off	Rise time	5p s	off
Fall time	5p s	off	Fall time	5p s	off
Pulse width	1/50M s	off	Pulse width	1/50M s	off
Temperature coefficient 1		off	Temperature coefficient 1		off
Temperature coefficient 2		off	Temperature coefficient 2		off
Nominal temperature		off	Nominal temperature		off
Type of rising & falling edge		off	Type of rising & falling edge		off

Fig 3.1: Vpulse for REF

Fig 3.2: Vpulse for DIV

The testbench should look like the one in Fig. 4

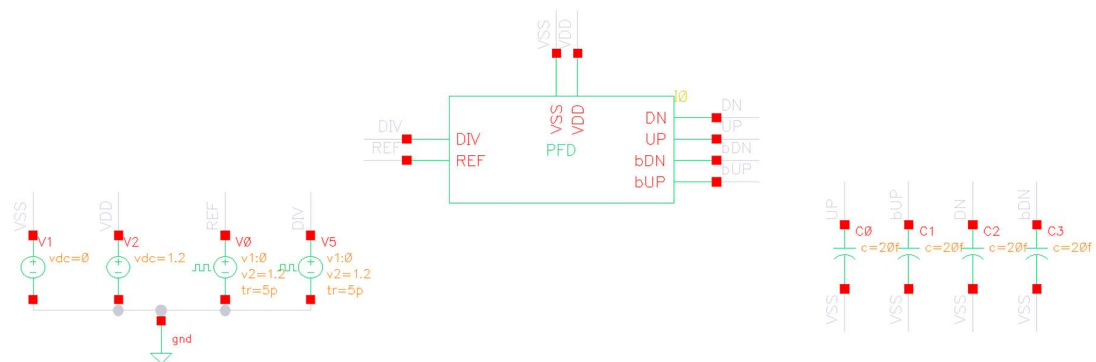


Fig 4: PFD testbench

We can now move to the testing part. From the schematic view you can create a new ADE Assembler view from *Launch > ADE Assembler > Create New View*. Here, create a new test, with a 50ns transient analysis in conservative mode.

In addition, to use standard cells in analog simulation, in ADE go to *Setup > Simulation Files*, In *Definition Files* add the CDL file for the standard cell library you want to use (for example) */usr/pack/tsmc-65-kgf/arm/tsmc/cln65lp/sc8_base_rvt/r0p0/cdl/sc8_cln65lp_base_rvt.cdl*

Check

http://eda.ee.ethz.ch/index.php?title=Tsmc65#Using_standard_cells_in_analog_simulation for more information regarding TSMC65 standard cells.

Set the parameter **dly_deg** to 10, run the analysis, and plot the IN and OUT waveforms.

10 degrees correspond to 1.11 ns

Q2.1: Please describe the functionality of the PFD.

Q2.2: What is the purpose of the Buffers and Delays (I12, I13, I14 in the picture above)? Why are those components needed?

Q2.3: Please elaborate on how to test the correct functionality of the instances mentioned in the question Q2.2. Do you need a specific input condition to test them?

3. Design of Charge Pump

Create a new cellview, and call it CH_PUMP, and draw the schematic depicted in Fig. 5. In this schematic, the charge pump is simplified by using some real current sources as source and sink currents, while switches connected to the UP, DN signals coming from the PFD dictate when current will be fed/withdrawn from the node VCP. Create a symbol.

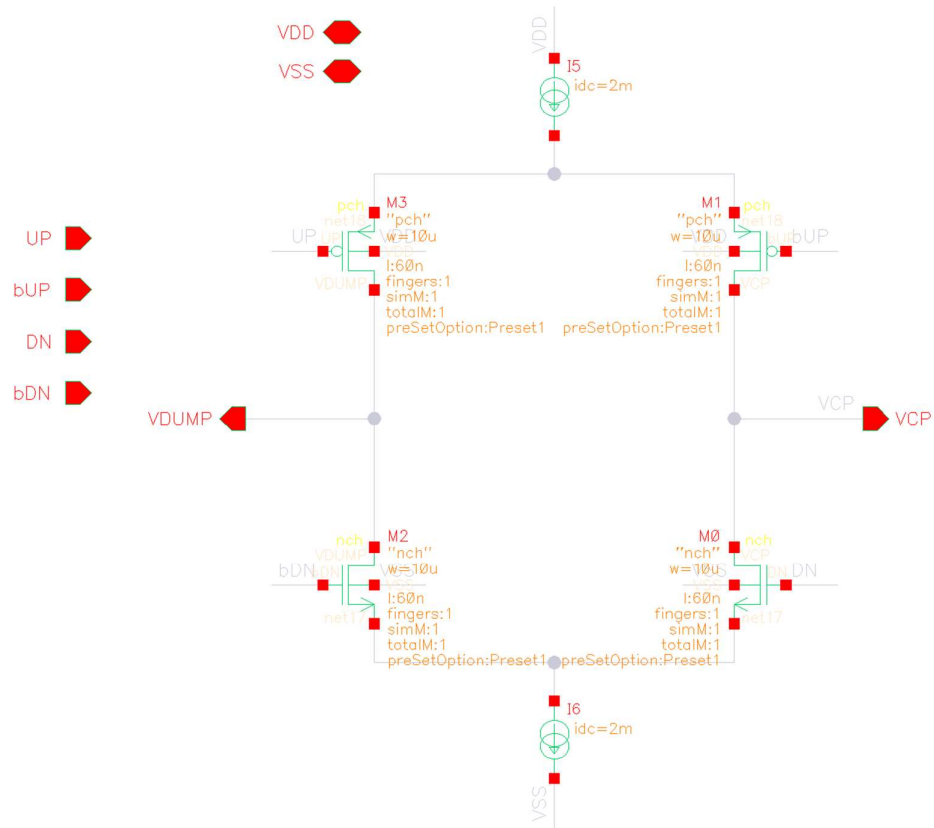


Fig. 5: Simplified schematic of charge pump

Q3.1: Why am I adding the dump branch connected to VDUMP? What do you think would be the drawback of not having it? (Consider the case with a real mirror, instead of the ideal current source used above).

As a next step we can verify the behaviour of the CP. The key behaviours that we want to check are correct DC operation, and sufficient noise performance. Create a new cellview called zTEST_CP, and instantiate the CP you just created in the schematic. We care about the behaviour while the CP is active, we will thus tie both UP and DN to VDD. Moreover, to set up a current noise analysis we need to instantiate a current probe (analogLib > iprobe) at the VCP node. The final schematic is shown in Fig. 6.

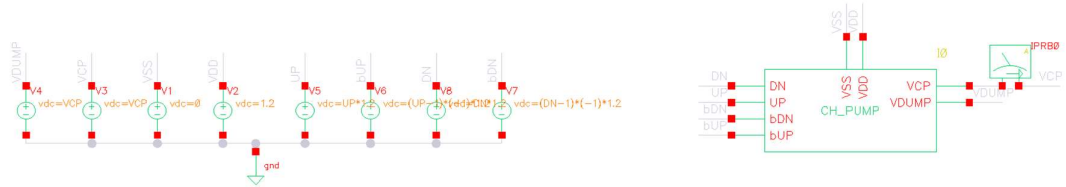


Fig.6: CP testbench

From here we can create a maestro view opening ADE Assembler as before, and will add a test with a *dc* and a *noise* analysis. The latter is setup as shown in Fig.7.

Fig.7: Noise analysis setup

Run the simulation.

The *dc* is used to check the operating points. You can annotate them on the schematic by right-clicking anywhere in the results window, and selecting *Annotate>DC Node voltages* and *Annotate>DC Operating Points*.

The output noise can be plotted by right clicking anywhere in the Results tab, and then *Direct Plot > Main Form*. The Main Form setup is shown in Fig. 8.

The 'Direct Plot Form' dialog box is shown with the following settings:

- Plotting Mode:** Append
- Analysis:** noise (selected)
- Function:** Output Noise (selected), Input Noise, Noise Figure, Noise Factor, Transfer Function
- Signal Level:** A / sqrt(Hz) (unselected), $A \cdot \sqrt{2} / \text{Hz}$ (selected)
- Modifier:** Magnitude (selected), dB20 (unselected)
- Add To Outputs:** checked
- Buttons:** Plot, OK, Cancel, Help
- Footer:** > Press plot button on this form...

Fig. 8: Main Form setup

Note how these results has very limited meaning, and it is just to show the procedure to verify the CP. The dc analysis will show perfectly matched currents, as we are using ideal sources, and the dominating noise source comes from the transistors that are here substituted by ideal current sources, leading to very low noise levels.

Now that we have a PFD and a rudimental CP we can test their functionality together. Create a zTEST_PFD_CP cellview . Then, open the schematic, and instantiate PFD and CP. Connect them. For now we will tie VCP and VDUMP at half VDD. In reality, the PLL will make sure that VCP will be locked at the value that will make the VCO lock to the correct target.

The final schematic should resemble the one in Fig.9.

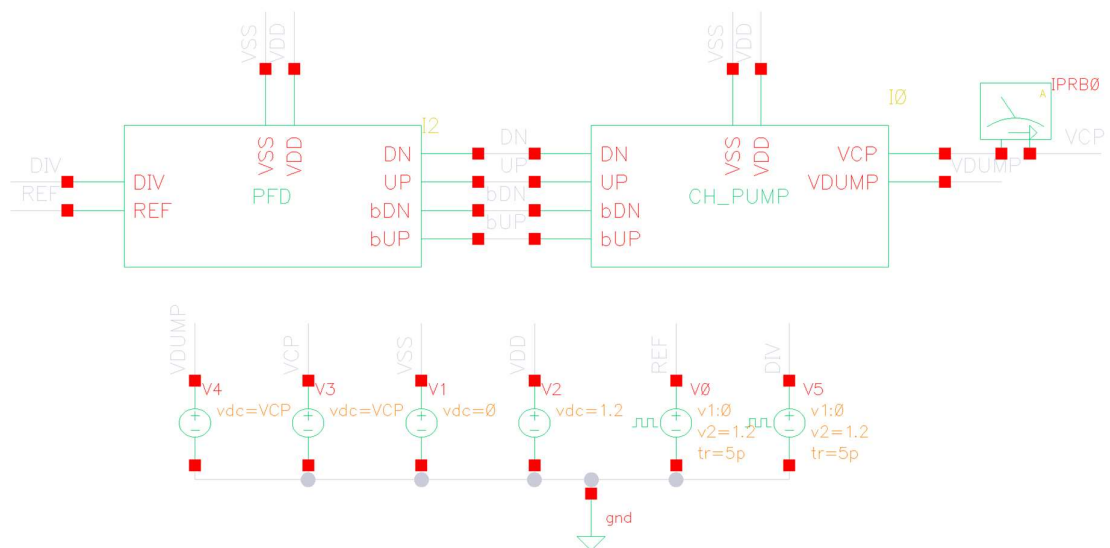


Fig. 9: Schematic of PFD+CP

Now that we also have the CP, we can simulate the transfer function as the overall charge coming from the CP (Q_{CP}) as a function of the phase error. To do so, first we have to add the current flowing through VCP to the outputs. Then, the total charge can be found with the calculator as:

$$\text{integ}(\text{clip}(\text{IT}("/I0/VCP") 0.5e-08 4.5e-08))$$

Careful: I0 is the name of my instance of the CP block. You should substitute it with your instance name.

Then, we can add the *dly_deg* variable to the Global Variables, and make it sweep from -tref to +tref by setting it to: *-180:10:180*. The resulting plot should resemble the one in Fig. 10.

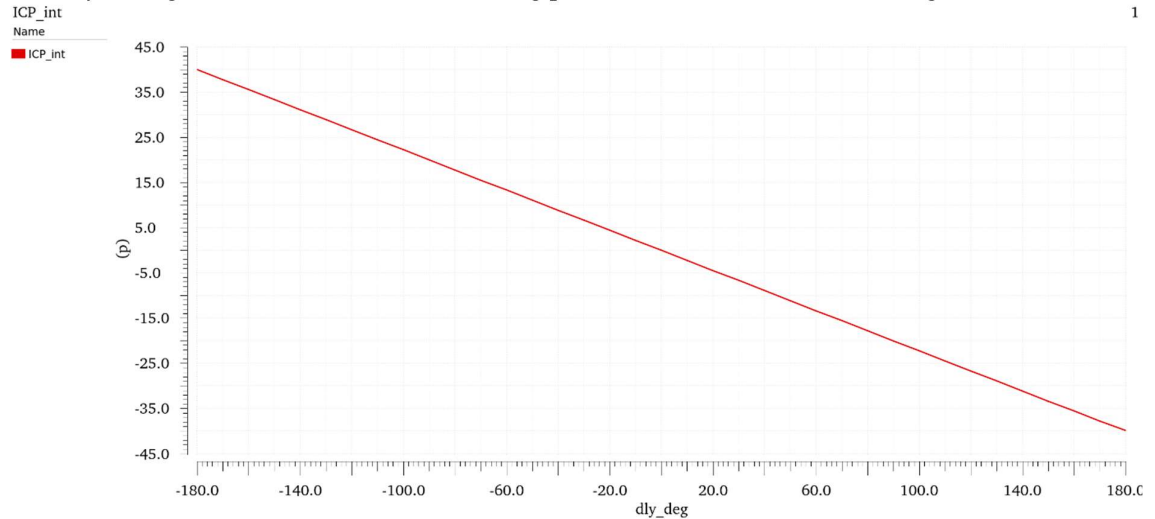


Fig. 10: Output of ideal CP

We can also check the absence of dead-zone by running a much finer sweep around the zero delay point by setting it to *-2:0.5:2*. The result is depicted in Fig. 11.

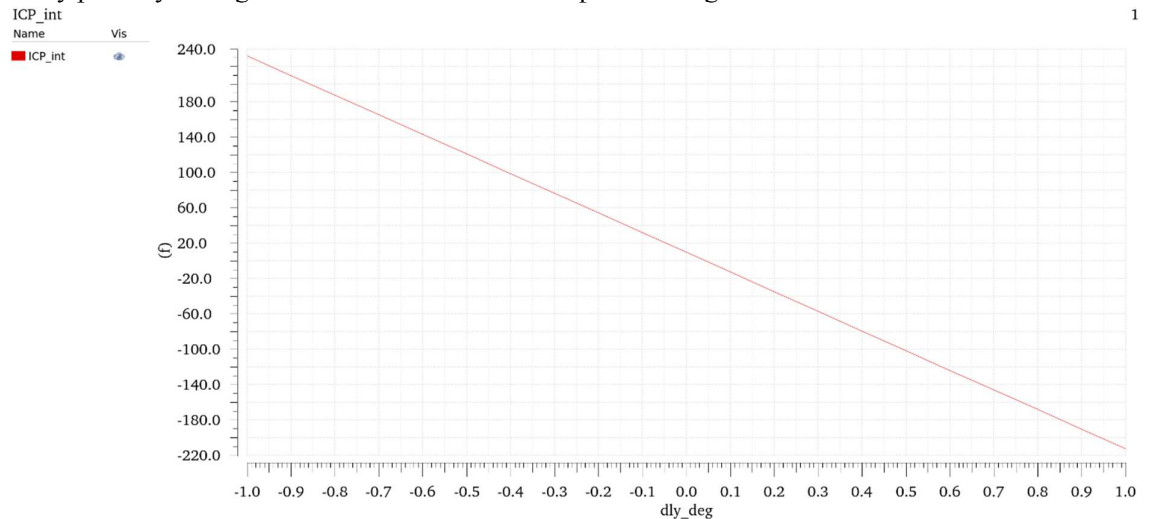


Fig.11: Dead-zone of ideal CP

As a next step we will calculate the noise contribution of the CP. To do so, for simplicity, we will create yet another TB, called *zTEST_CP_noise*, this time removing the PFD. Moreover, the CP will inject noise when the main branch is active. For this reason, we will keep both UP and DN tied to VDD for this analysis.

4. Design of real CP

Now you will apply what you have learnt so far to design a charge pump without ideal blocks. The three biggest issues that will arise the moment in which you will replace the ideal sources with mirrors are source and tail matching, CP noise, and flicker noise.

A4.1: Design a CP with 2mA of nominal current generated from a single ideal current source of 20uA, that meets the following specs:

1. Max UP-DN current mismatch of 0.5% (10uA)
2. Mismatch constraint respected at different VCP voltages (0.4V, 0.6V, 0.8V)
3. Max current consumption of the whole block 2.3mA
4. Noise thermal floor below $1e-21 \text{ V}^2/\text{Hz}$
5. Flicker corner before 50kHz

Hint: a simple current mirror will probably not be enough. Do you know techniques for increasing the impedance of the output branch from the AIC course?

Hint2: The diode connected side of the mirrors would also contribute to the output noise, however we can filter them out with a simple RC. Making a large C is expensive, as it would require a large amount of area, and it is thus impractical. On the other hand, a large R can be implemented with a much smaller area investment using pseudo-resistors, or duty cycled resistors. However, a large R would lead to a large voltage drop due to gate leakage. This will create a large mismatch between source and tail currents, and should thus be avoided. The easiest solution to this is to simply design the mirrors with thick gate transistors (nch_25 and pch_25), which have much smaller gate leakage. Note that you should have just enough headroom with $V_{DD}=1.2V$, so you do not need to increase it.

A4.2: What are the biggest noise contributors? How would you minimize their contribution? What are the tradeoffs? Explain qualitatively your design choices for the key transistors of the block.

A4.3: What is the V_{ov} needed to achieve the target noise floor? (Consider $\gamma=2/3$)

A4.4: Up to now we are connecting VCP and VDUMP to the same potential artificially in the TB. This, however, cannot be done in reality, as we do not know the exact voltage at which the PLL will lock beforehand. What do you think will happen if VDUMP and VCP are not matched? How will it reflect on the PLL? What could you add to the circuit to make sure that the two nodes are matched? Explain qualitatively.